🧠 Ejercicios de JavaScript

A continuación se presentan 10 ejercicios para practicar el uso de funciones, arreglos, objetos, desestructuración y promesas básicas con `setTimeout`.

---

1. 🔁 Agrupar elementos por propiedad

Dado un arreglo de objetos, agrúpelos por el valor de una propiedad específica.

```
const datos = [
  { categoria: 'fruta', nombre: 'manzana' },
  { categoria: 'verdura', nombre: 'zanahoria' },
  { categoria: 'fruta', nombre: 'banana' },
  { categoria: 'verdura', nombre: 'lechuga' }
];

const resultado = agruparPorPropiedad(datos, 'categoria');
console.log(resultado);
// Resultado esperado:
// {
//   fruta: [
//     { categoria: 'fruta', nombre: 'manzana' },
//     { categoria: 'fruta', nombre: 'banana' }
//   ],
//   verdura: [
//     { categoria: 'verdura', nombre: 'zanahoria' },
//     { categoria: 'verdura', nombre: 'lechuga' }
//   ]
// }
```

---

2. ⛓️ Promesas encadenadas

Escribe una función `obtenerUsuario(id)` que simule una llamada asíncrona con `setTimeout` y retorne una promesa que se resuelve con un objeto `{ id, nombre }`. Encadena dos llamadas para obtener los nombres de dos usuarios diferentes y mostrar el mensaje:

```
"Usuarios cargados: Juan y María"
```

---

3. 🧱 Calcular promedio ponderado

Dado un arreglo de objetos con valores y pesos, calcula el promedio ponderado.

```
const notas = [
  { valor: 90, peso: 0.5 },
  { valor: 80, peso: 0.3 },
  { valor: 70, peso: 0.2 }
];

const resultado = promedioPonderado(notas);
console.log(resultado);
// Resultado esperado: 83
```

🔎 ¿Qué significa “peso”?

El **peso** es la **importancia o porcentaje** que representa cada nota en el promedio final. Por ejemplo:

- `90` con peso `0.5` representa el 50% de la nota.
- `80` con peso `0.3` representa el 30%, etc.

✅ ¿Cómo se calcula el promedio ponderado?:

```
Multiplicas cada nota por su peso, sumas todo y ese es el promedio.
```

---

4. 🎭 Copiar y modificar objetos

Crea una función que reciba un objeto `persona` y retorne una copia modificada con un nuevo nombre y la edad aumentada en 1 año, sin modificar el objeto original.

```
const persona = { nombre: 'Luis', edad: 30 };

const nuevaPersona = modificarPersona(persona, 'Carlos');
console.log(nuevaPersona);
// Resultado esperado: { nombre: 'Carlos', edad: 31 }
```

---

5. 🔁 Eliminar duplicados por propiedad

Escriba una función que reciba un arreglo de objetos y elimine duplicados basándose en una propiedad específica.

```
const elementos = [
  { id: 1, nombre: 'A' },
  { id: 2, nombre: 'B' },
  { id: 1, nombre: 'A' }
];

const resultado = eliminarDuplicados(elementos, 'id');
console.log(resultado);
// Resultado esperado: [{ id: 1, nombre: 'A' }, { id: 2, nombre: 'B' }]
```

---

6. 🧠 Combinar dos listas sin repetir elementos

Crea una función que reciba dos arreglos y devuelva un nuevo arreglo que contenga todos los elementos **sin repetir**.

```
const lista1 = ['manzana', 'pera', 'banana'];
const lista2 = ['pera', 'uva', 'manzana', 'sandía'];

const resultado = combinarListasSinRepetir(lista1, lista2);
console.log(resultado);
// Resultado esperado: ['manzana', 'pera', 'banana', 'uva', 'sandía']
```

---

7. 📦 Contar cuántas veces aparece cada elemento

Crea una función que reciba un arreglo de palabras y retorne un objeto donde las claves sean las palabras y los valores la cantidad de veces que aparecen.

```
const palabras = ['hola', 'adiós', 'hola', 'gracias', 'hola',
'gracias'];

const resultado = contarPalabras(palabras);
console.log(resultado);
// Resultado esperado: { hola: 3, adiós: 1, gracias: 2 }
```

---

8. 🎯 Filtrar objetos por múltiples condiciones

Crea una función que reciba una lista de productos y devuelva solo los productos que sean de cierta categoría y que tengan stock disponible.

```
const productos = [
  { nombre: 'Camisa', categoria: 'ropa', stock: 10 },
  { nombre: 'Pantalón', categoria: 'ropa', stock: 0 },
  { nombre: 'Televisor', categoria: 'electrónica', stock: 5 }
];

const resultado = filtrarDisponibles(productos, 'ropa');
console.log(resultado);
// Resultado esperado: [{ nombre: 'Camisa', categoria: 'ropa', stock: 10
}]
```

---

9. 🕰️ Simular carga con promesa y setTimeout

Crea una función que simule una carga de datos usando `setTimeout` y devuelva una promesa que se resuelve después de 2 segundos con el mensaje: "Datos cargados correctamente".

```
cargarDatos().then((mensaje) => {
  console.log(mensaje);
});
// Resultado esperado (después de 2 segundos): "Datos cargados
correctamente"
```

---

10. 🔍 Verificar si todos los elementos cumplen una condición

Crea una función que reciba una lista de edades y verifique si **todos** los elementos son mayores de edad (18 años o más).

```
const edades = [19, 25, 32, 17];

const resultado = verificarMayoresDeEdad(edades);
console.log(resultado);
// Resultado esperado: false
```
